

Process

PROCESS

- A process is an **instance** of a **computer program** that is being executed.
- A program becomes a process when an executable file is **loaded into memory**.
- Operating systems have **many background tasks running**, your computer is likely to have many more processes running than actual programs.

PROCESS (CONT.)

- For example;

You may only have **three programs running**, but there may be **twenty active processes**.

You can view active processes in Windows by opening the **Task Manager** (press Ctrl-Alt-Delete and click Task Manager).

PASSIVE ENTITY & ACTIVE ENTITY

- File containing list of **instructions stored on disk** (often called executable file) called passive entity.
- A **process** is an **active entity**, with a program counter specifying the next instruction to execute and a set of resources.

A PROCESS GENERALLY INCLUDES

1. Process Stack

Process stack contain **temporary data** (such as function parameters, return addresses and local variables)

2. Data Section

Data section contains **global variables**.

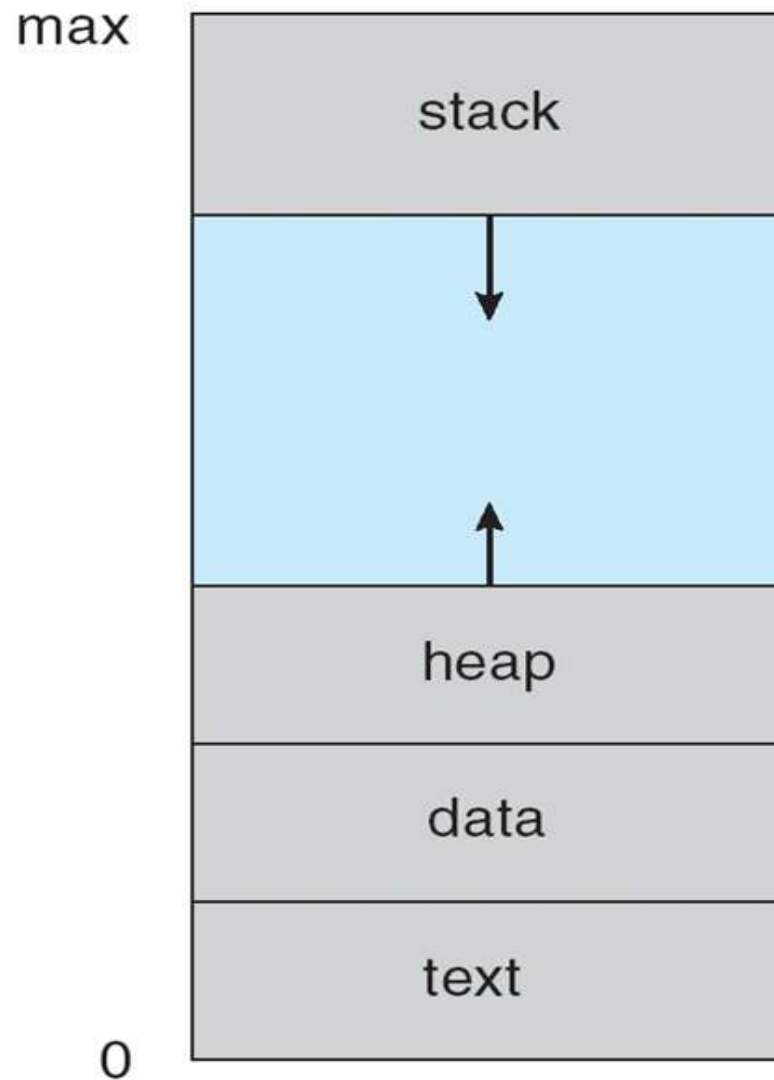
3. Heap

Heap is a **memory** that is **dynamically allocated** during process run time.

4. Text Section

Contains **program code**.

PROCESS IN MEMORY

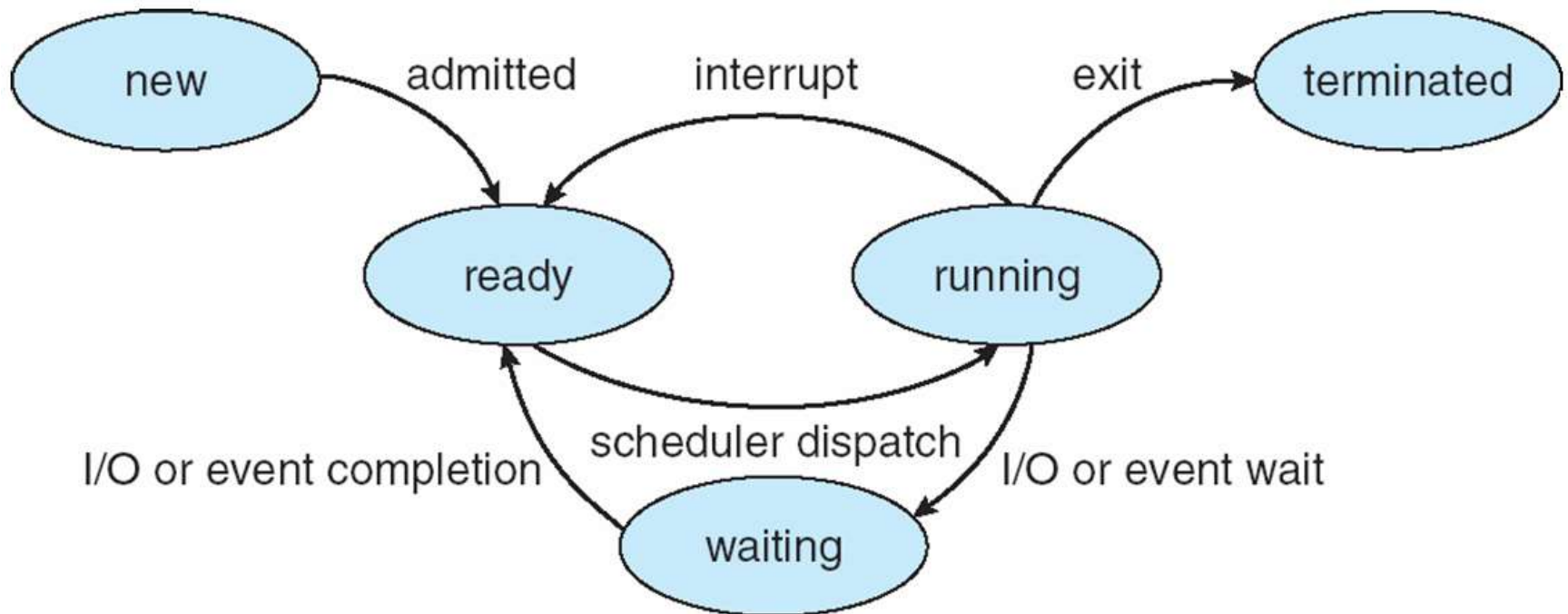


PROCESS STATES

As a process executes, it **changes state**. Each process may be in one of the following states.

- **New**
The process is being **created**.
- **Running**
Instructions are being **executed**.
- **Waiting**
The **process is waiting** for some event to occur(such as an I/O completion or reception of a signal).
- **Ready**
The process is **waiting to be assigned** to a processor.
- **Terminated**
The process has **finished execution**.

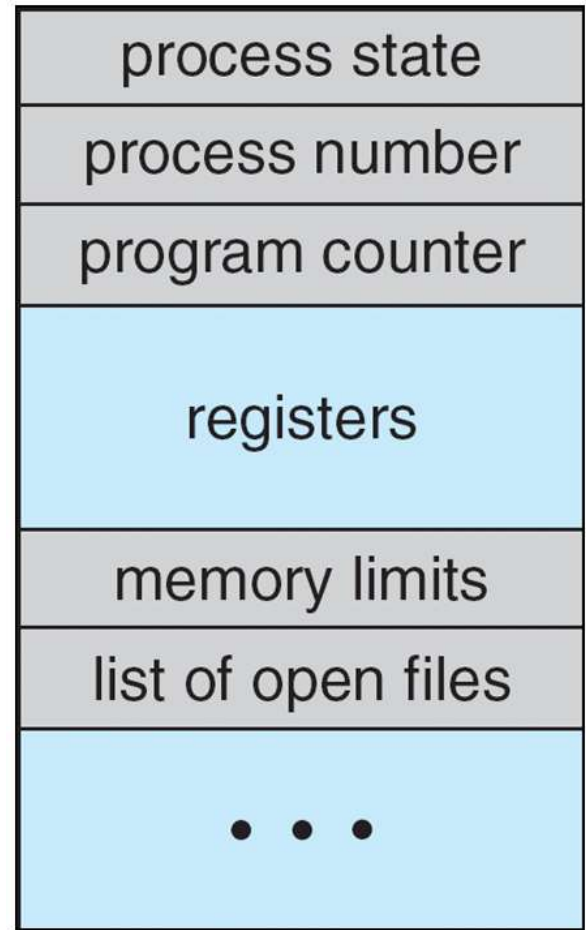
PROCESS STATES DIAGRAM



Process Control Block (PCB)

Each **process** is represented in **Operating System** by a **Process Control Block(PCB)**.

- It is also called a **task control block**



Process Control Block (cont.)

Process state

- The state may be **new**, **ready**, **running**, **waiting** and so on.

Program counter

- Program counter indicate the **address of the next instruction** to be **executed** for this process.

Process Control Block (cont.)

CPU registers

The registers vary in **number** and **type**, depending on the computer architecture. They include:

- Accumulators(i.e an accumulator is a register in which intermediate **arithmetic** and **logic** results are stored).
- Index registers(i.e index register used for **modifying operand addresses** during the run of a program).
- Stack pointers(The stack pointer is used **to give the address of the stack area**, where the local variables and parameters to subroutines are stored).
- General purpose registers(i.e general purpose register is a **small amount of storage** available on the CPU whose contents can be accessed more quickly than storage available elsewhere).

Process Control Block (cont)

- **CPU-scheduling information**

This information includes process **priority**, **pointers to scheduling queues**, and any other scheduling parameters.

- **Memory-management information**

This includes value of the **base and limit registers**, the **page tables**, or the **segment tables**.

Process Control Block (cont)

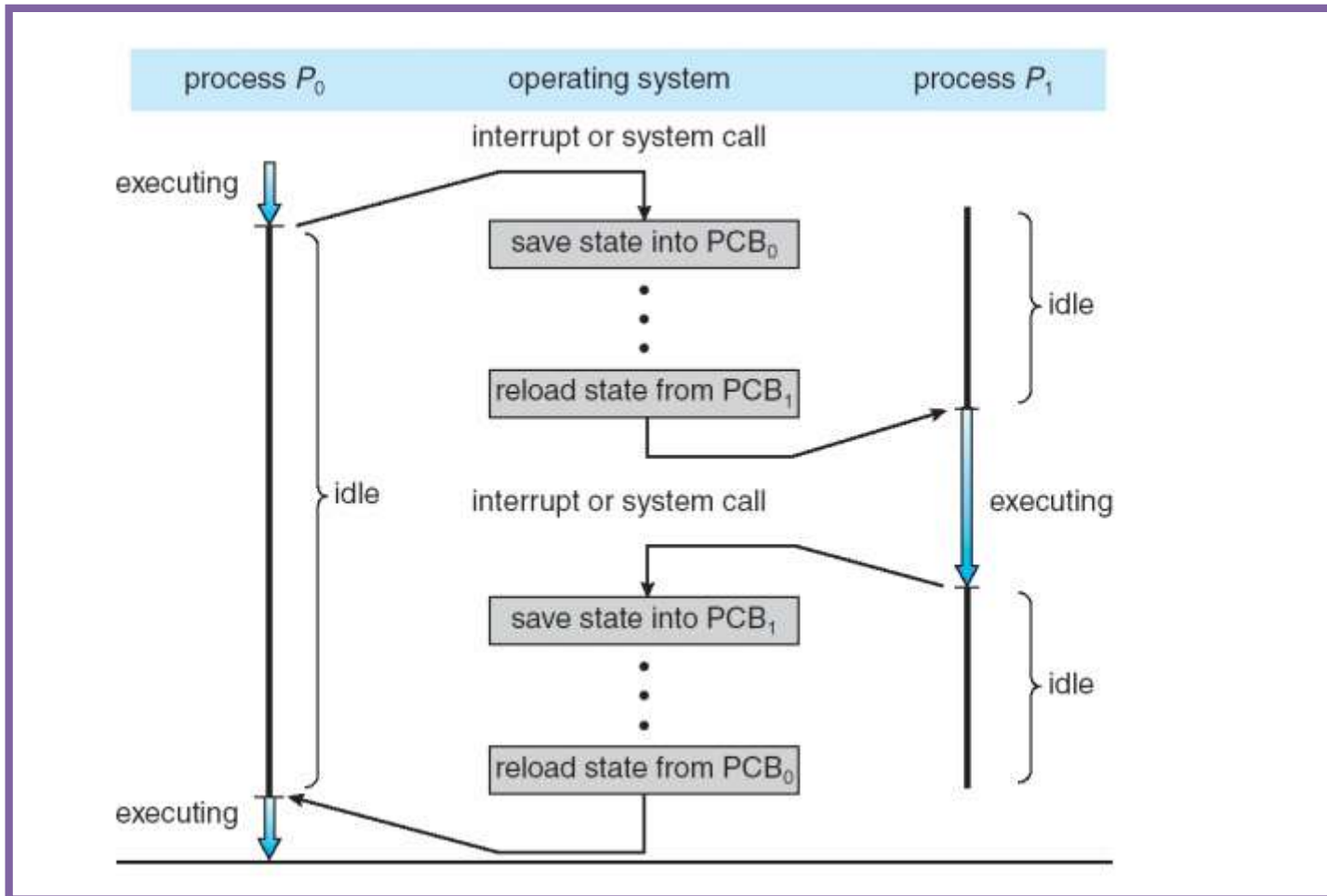
- **Accounting information**

This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers and so on.

- **I/O status information**

This includes the list of I/O devices allocated to the process, a list of open files and so on.

CPU Switch From Process to Process



Time Sharing

The objective of time sharing is to;

- Switch the CPU among **processes** so frequently that users can interact with each program while it is running.

Process Scheduler & Process Identifier

- **Process Scheduler**

The purpose of process scheduler is to **selects** an **available process** (possibly from a set of several processes) for program **executing on the CPU**.

- **Process Identifier (pid)**

Process identifier is used to **uniquely identify a process**.

OPERATIONS ON PROCESSES

There are **two main operations** that are applied on process.

1. **Process Creation**
2. **Process Termination**

PROCESS CREATION

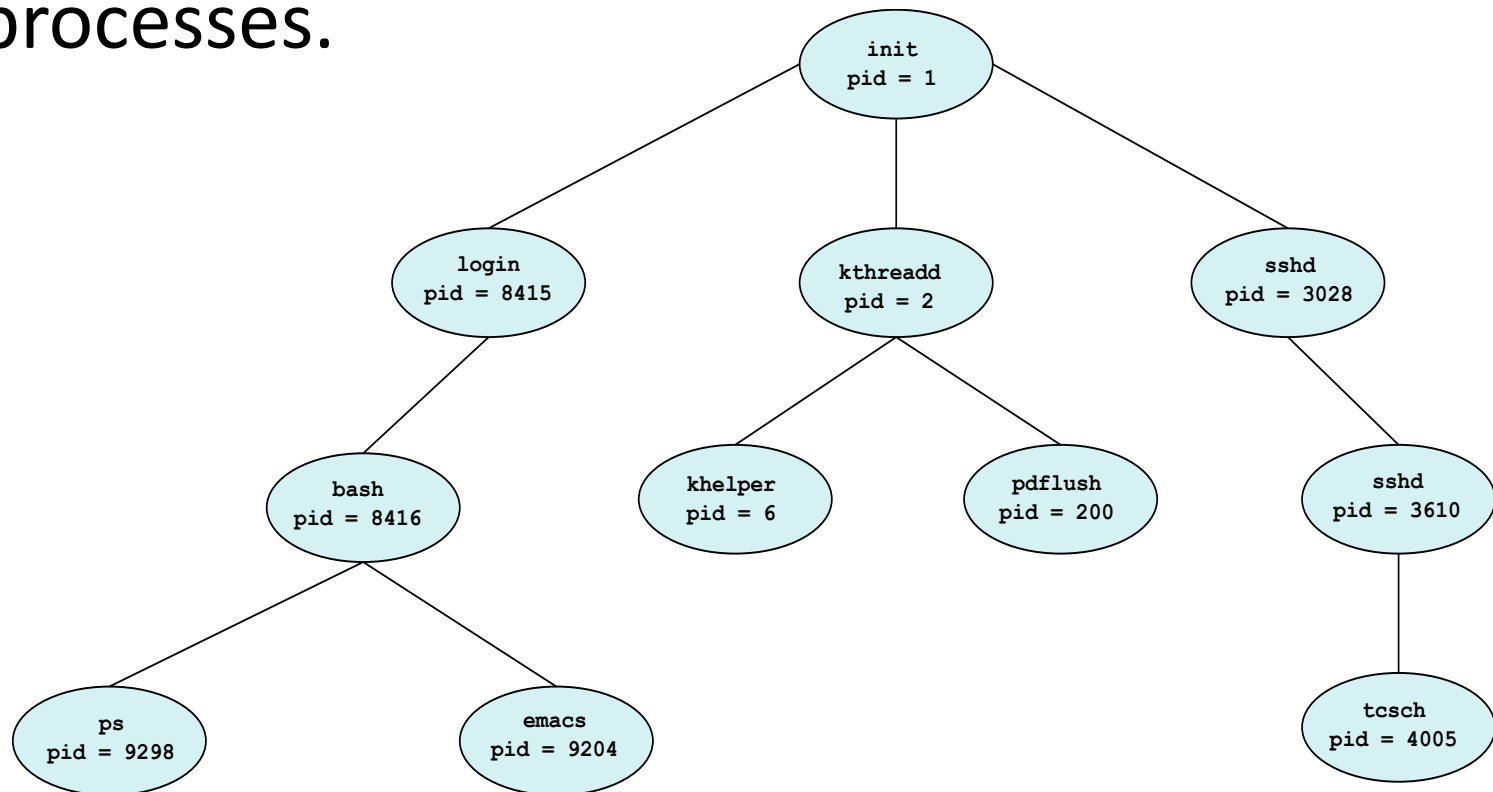
A process may **create several new processes**, via a create- process system call, during the course of execution.

Parent Process: The creating process is called parent process.

Child process: The new process is called the children of that process.

PROCESS CREATION (cont.)

Each of these new process may inturn create other processes, which forming a tree of processes.



PROCESS TERMINATION

A process **terminate** when it **finishes executing its final statement** and ask the operating system to delete it by using the `exit()` system call.

THREE MAJOR ACTIVITIES OF AN OPERATING SYSTEM IN REGARD TO MEMORY MANAGEMENT

Operating System has following three major activities in regard to memory management.

1. Keep track of which part of memory are currently being used and by whom.
2. Decides which processes are to be loaded into memory when memory space becomes available.
3. Allocate and de-allocate memory space as needed.

THREE MAJOR ACTIVITIES OF AN OPERATING SYSTEM IN REGARD TO SECONDARY STORAGE MANAGEMENT

Operating System has following three major activities in regard to secondary storage management

- Free space management
- Storage management
- Disk Scheduling

SCHEDULING QUEUES

As processes enter the system, they are put into a job queue, which consists of all processes in the system.

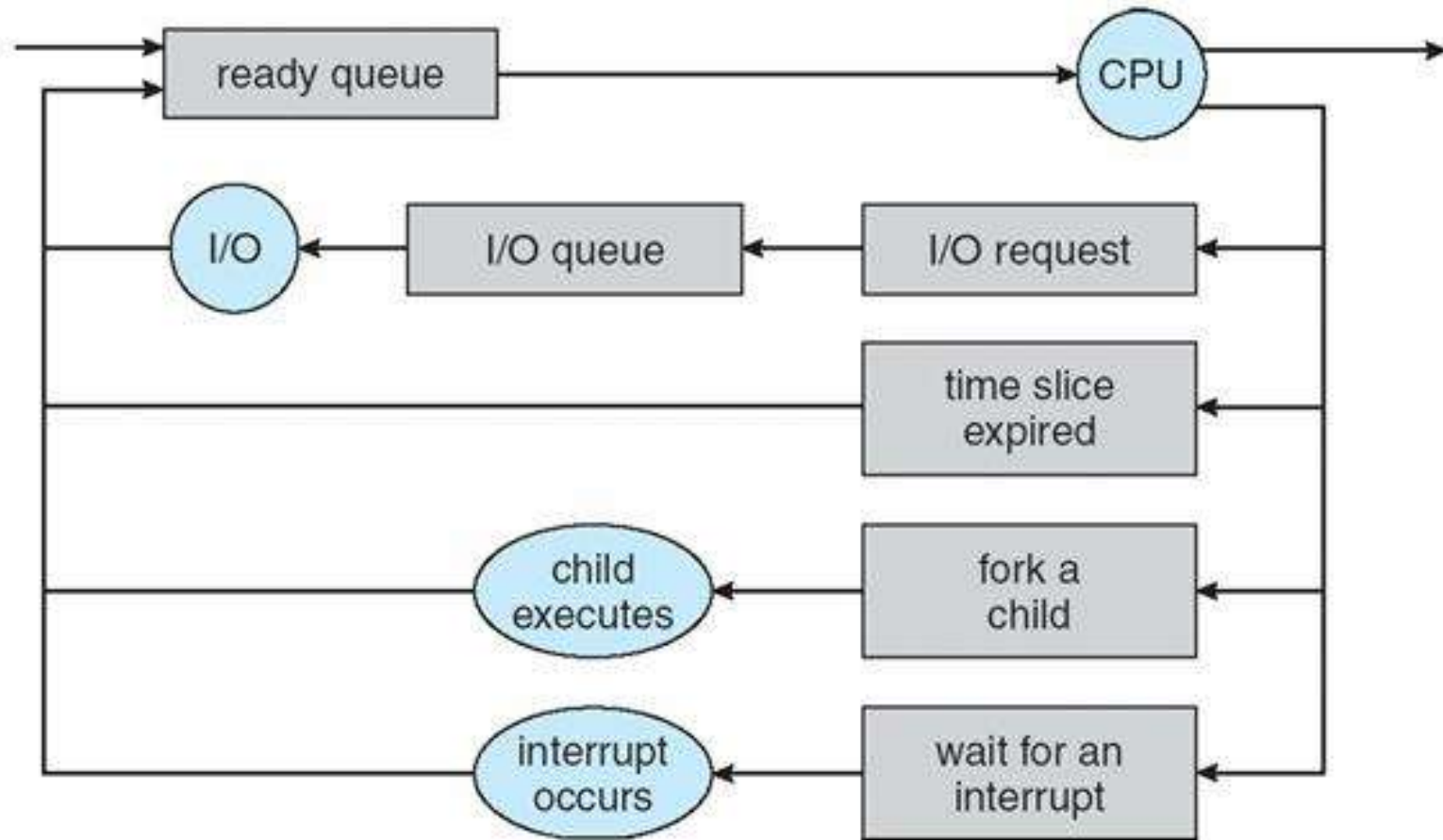
- **Ready Queue**

The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the ready queue.

- **Device Queue**

Suppose the process makes an I/O request to a shared device, such as a printer. Since there are many processes in the system, the printer may be busy with the input/output request of some other process. The process may have to wait for the printer. This list of processes waiting for a particular I/O device is called device queue.

SCHEDULING QUEUES(CONT.)



WHICH EVENTS COULD OCCUR WHEN A PROCESS IS ALLOCATED THE CPU AND IS EXECUTING

When a process is executing , then following events can occurs.

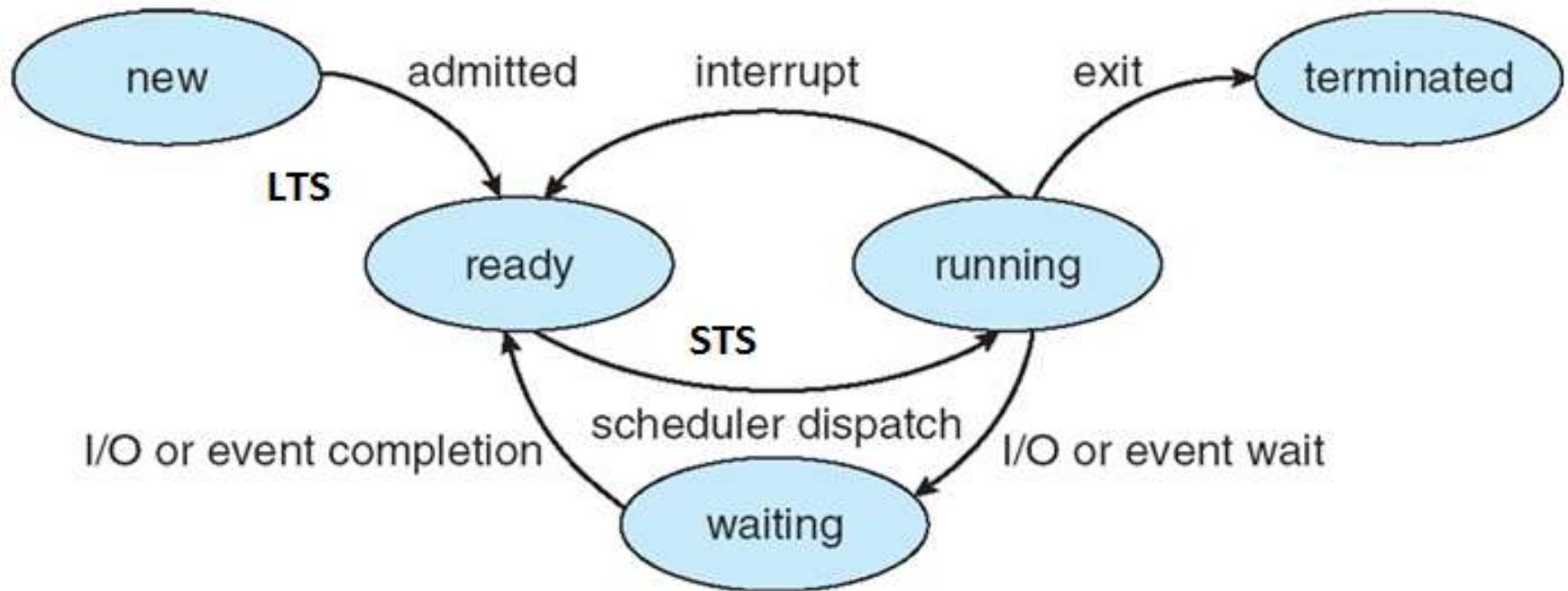
1. The process could issue an I/O request and then be placed in an I/O queue.
2. The process could create a new sub-process and wait for the sub-process's termination.
3. The process could be removed by force from the CPU, as a result of an interrupt, and be put back in the ready queue.

SCHEDULERS

There are three types of schedulers in Operating System

- Long Term Scheduler (or job scheduler)
selects which processes should be brought into the ready queue.
New state to ready state.
- Short Term Scheduler (or CPU Scheduler)
selects which process should be executed next and allocates CPU.
Ready state to running state.
- Medium Term Scheduler (or swapper)
responsible for suspending and resuming the processes. Main
memory to Secondary memory.

Process State Diagram



Degree of Multiprogramming

- The no. of processes present in the main memory at any point of time is called degree of multiprogramming.
- Long Term Scheduler controls the degree of multiprogramming.
- LTS should select the good combination of CPU bound and I/O bound Process in order to get good throughput.
- If more CPU bound Process then CPU utilization more and throughput low.

I/O-bound process

- Spends more time doing I/O than computations, short CPU bursts

CPU-bound process

- Spends more time doing computations; very long CPU bursts

CPU Burst

- a time period during which the process wants to continuously run in CPU.

Cooperating Processes

- Processes within a system may be independent or cooperating
 - **Independent** process cannot affect (or be affected) by the execution of another process
 - **Cooperating** process can affect (or be affected) by the execution of another process
- Advantages of process cooperation
 - Information sharing
 - Computation speed-up
 - Modularity (application will be divided into modules/sub tasks)
 - Convenience (may be better to work with multiple processes)

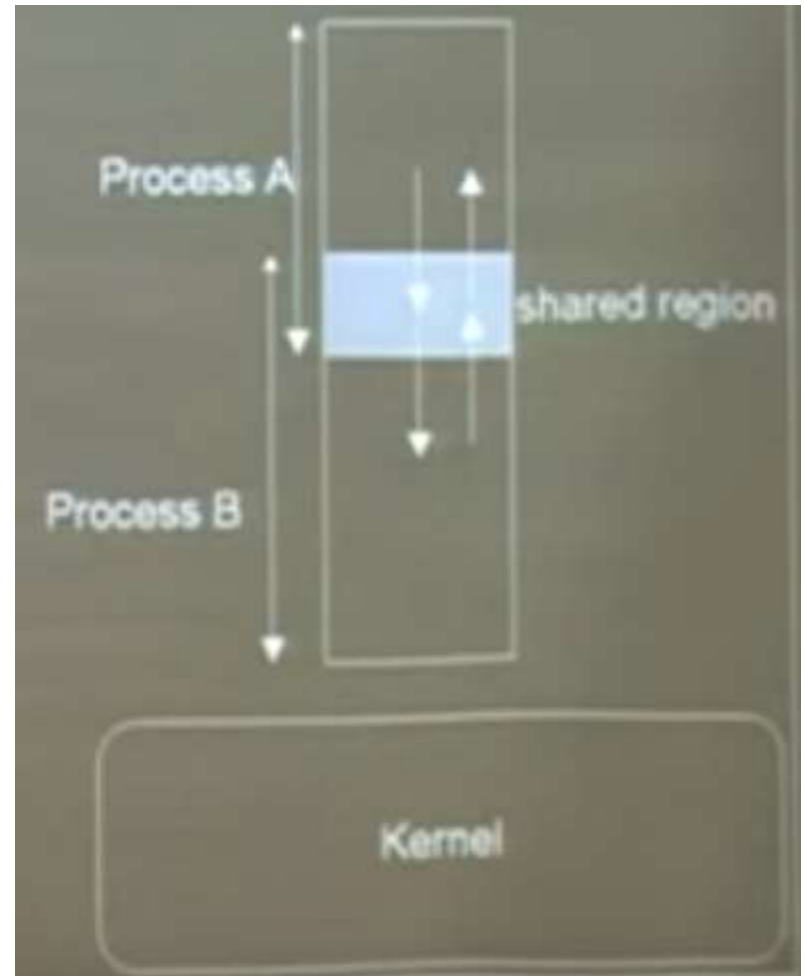
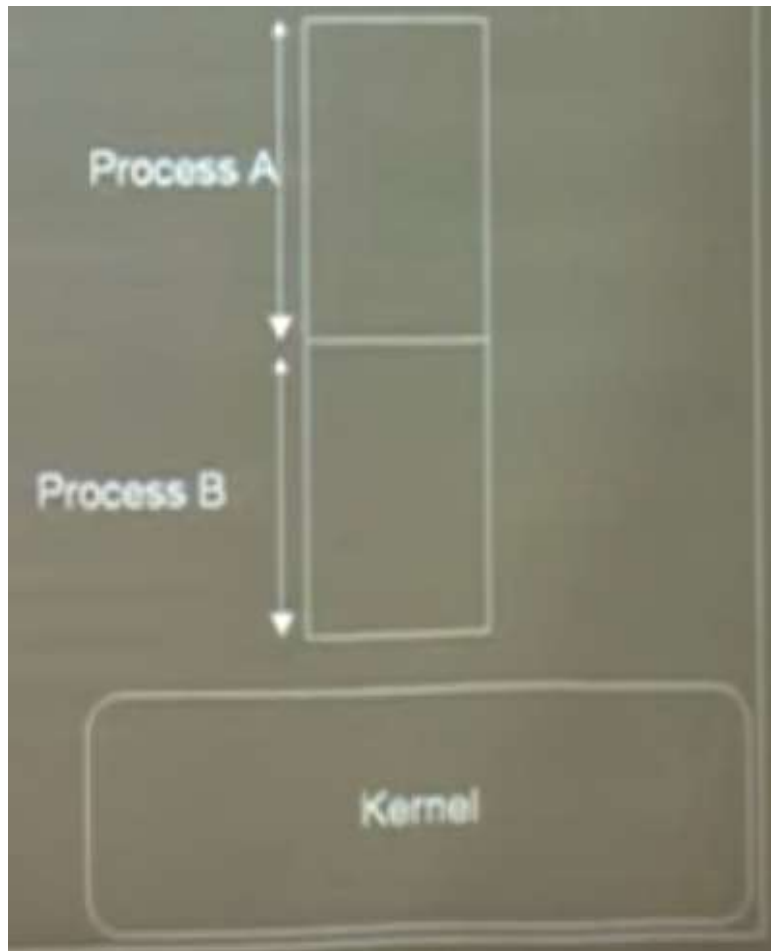
Inter Process Communication(IPC) Mechanisms

- Cooperating process require a facility/mechanism for interprocess communication(IPC)
- There are two basic mechanism provided by most systems
 1. Shared Memory
 2. Message Passing

Shared Memory IPC Mechanism

- A region of shared memory is established between (among) two or more processes
- Establishment of that shared region is done via the help of the operating system kernel
- Then, processes can read and write shared memory region (segment) directly as ordinary memory access (like they are accessing memory variables directly without kernel help)

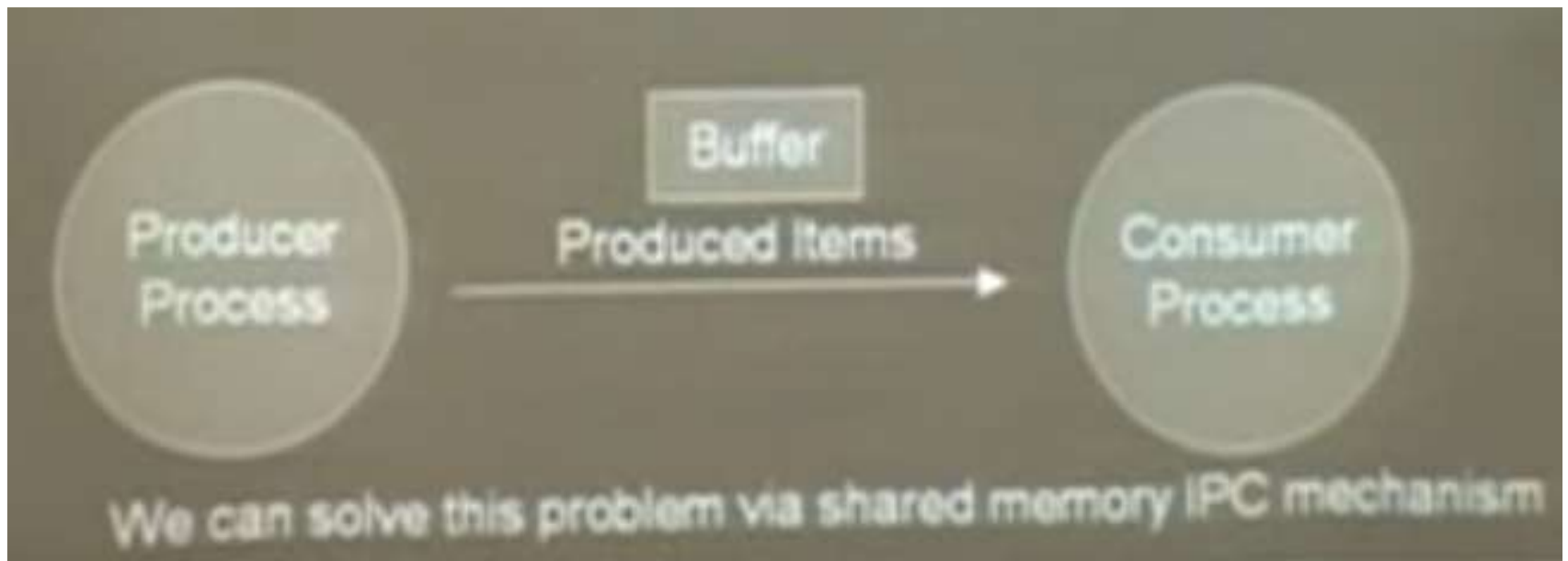
Shared Memory IPC Mechanism



Shared Memory IPC Mechanism

- To illustrate the use of the shared memory IPC mechanism, a general model problem, called producer-consumer problem, can be used.
- **Producer-consumer problem**
we have a producer, a consumer and data is sent from producer to consumer.
 - unbounded-buffer places no practical limits on the size of the buffer
 - Bounded-buffer assumes that there is fixed buffer size

Producer-Consumer Problem



Producer-Consumer Problem



Buffer



Producer



Consumer

Producer

```
while (1) {  
    /* produce an item in nextProduced */  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in+1) % BUFFER_SIZE;  
    counter++;  
}
```

Consumer

```
while (1) {  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out+1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in nextConsumed */  
}
```



counter



register₁



register₂

Although both the producer and the consumer routines are correct separately, they may not function correctly when executed.

Producer-Consumer Problem

Producer

```
while (1) {  
    /* produce an item in nextProduced */  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in+1) % BUFFER_SIZE;  
    counter++;  
}
```

Consumer

```
while (1) {  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out+1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in nextConsumed */  
}
```

5

counter



register₁



register₂

Suppose that the value of the variable "counter" is currently 5 and that the producer and consumer processes execute the statements "counter++" and "counter--" concurrently. The only correct result would be, after execution of the producer and the consumer, "counter = 5"

Producer

```
while (1) {
    /* produce an item in nextProduced */
    while (counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in+1) % BUFFER_SIZE;
    counter++;
}
```

Counter++

```
register1 = counter
register1 = register1 + 1
counter = register1
```

Counter--

```
register2 = counter
register2 = register2 - 1
counter = register2
```

Above are machine language implementations of "counter++" and "counter--"

Consumer

```
while (1) {
    while (counter == 0)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out+1) % BUFFER_SIZE;
    counter--;
    /* consume the item in nextConsumed */
}
```

5

counter

register₁

register₂

Counter++

```
register1 = counter
register1 = register1 + 1
counter = register1
```

Counter--

```
register2 = counter
register2 = register2 - 1
counter = register2
```

Observe as these statements are interleaved.

5

counter

5

register₁

register₂

Counter++

```
register1 = counter
register1 = register1 + 1
counter = register1
```

Counter--

```
register2 = counter
register2 = register2 - 1
counter = register2
```

T₀: producer register₁ = counter
T₁: producer register₁ = register₁ + 1

5

counter

6

register₁

register₂

Counter++

```
register1 = counter
register1 = register1 + 1
counter = register1
```

Counter--

```
register2 = counter
register2 = register2 - 1
counter = register2
```

T₀: producer register₁ = counter
T₁: producer register₁ = register₁ + 1
T₂: consumer register₂ = counter

5

counter

6

register₁

5

register₂

Observe as these statements are interleaved.

Observe as these statements are interleaved.

Counter++

register₁ = counter
register₁ = register₁ + 1
counter = register₁

Counter--

register₂ = counter
register₂ = register₂ - 1
counter = register₂

T₀: *producer* register₁ = counter
T₁: *producer* register₁ = register₁ + 1
T₂: *consumer* register₂ = counter
T₃: *consumer* register₂ = register₂ - 1

Observe as these statements are interleaved.

5

counter

6

register₁

4

register₂

Counter++

register₁ = counter
register₁ = register₁ + 1
counter = register₁

Counter--

register₂ = counter
register₂ = register₂ - 1
counter = register₂

Observe as these statements are interleaved.

6

counter

6

register₁

4

register₂

Counter++

register₁ = counter
register₁ = register₁ + 1
counter = register₁

Counter--

register₂ = counter
register₂ = register₂ - 1
counter = register₂

T₀: *producer* register₁ = counter
T₁: *producer* register₁ = register₁ + 1
T₂: *consumer* register₂ = counter
T₃: *consumer* register₂ = register₂ - 1
T₄: *producer* counter = register₁
T₅: *consumer* counter = register₂

Observe as these statements are interleaved.

4

counter

6

register₁

4

register₂

Message Passing IPC Mechanism

- Mechanism for processes to communicate and synchronize their actions
- This IPC facility provides two operations:
 - Send (message)- message size fixed or variable
 - Receive (message)
- If P and Q wish to communicate, they need to:
 - establish a (logical) communication link between them.
 - Exchange messages via send/receive

Message Passing IPC Mechanism



Direct Communication

- Each process wanting to communicate must explicitly name the recipient or sender of communication.
 - **send**(destination-process P , $message$) – send a message to process P
 - **receive**(source-process Q , $message$) – receive a message from process Q
- Properties of a direct communication link
 - Links are established automatically
 - Link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- Messages are directed and received from mailboxes
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Properties of an indirect communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional

Indirect Communication

- Operations
 - create a new mailbox
 - send and receive messages through mailbox
 - delete a mailbox
- Primitives are defined as:
 - send**(*A, message*) – send a message to mailbox *A*
 - receive**(*A, message*) – receive a message from mailbox *A*

Synchronization

- Message passing may be either
 - Blocking, or
 - Non-blocking

- **Blocking** is considered **synchronous(occurring at same time)**
 - **Blocking send** -- the sender is blocked until receiving process does a corresponding read
 - **Blocking receive** -- the receiver is blocked until a message is sent for it to receive
- **Non-blocking** is considered **asynchronous(not occurring at same time)**
 - **Non-blocking send** -- the sender sends the message and continues
 - **Non-blocking receive** -- the receiver receives:
 - A valid message, or
 - Null message

Buffering in Message-Passing

- Messages exchange by communicating processes reside in a temporary queue.
- Queue implemented in one of three ways
 - 1. Zero capacity – queue length 0**

No messages wait. Sender must wait until receiver receive the message
 - 2. Bounded capacity – finite length queue**

Messages can be queued as long as queue not full, otherwise sender has to wait if queue full
 - 3. Unbounded capacity – infinite length**

Any number of messages can be queued ,sender never waits